LOW-LEVEL DESIGN · FLOW DIAGRAMS

Cold Storage Simulation

How the simulation runs from start to finish: the order of steps, who makes each decision, and how each controller works, from scheduling and physics to the controllers, AI planner and digital twin.

3 chambers | 5 controllers | 12-h economic MPC | Self-learning AI | EKF digital twin | Parallel benchmarks

0 Diagram legend

1 System context – layers

2 Startup flow – setup(seed)

3 Main loop – frame() (per browser animation frame)

4 tick(dt = 1 min) – master sequence

5 Operations & scheduling

6 Control layer – stepCompressor(i, dt, st, md)

7 Plant physics – stepZonesAndLearn → stepZone(i)

8 AI learner – learnerObserve (mode "ai" only)

9 Digital twin – sensing, estimation, anomaly, maintenance

10 Forecast & advisories

11 Metering, billing & benchmarking

12 External integration – window.ColdStorageTwin

13 Decision-maker register

14 Key constants (tuning surface)

Source
simulation/cold_storage_simulation.html · main @ 36f67d7

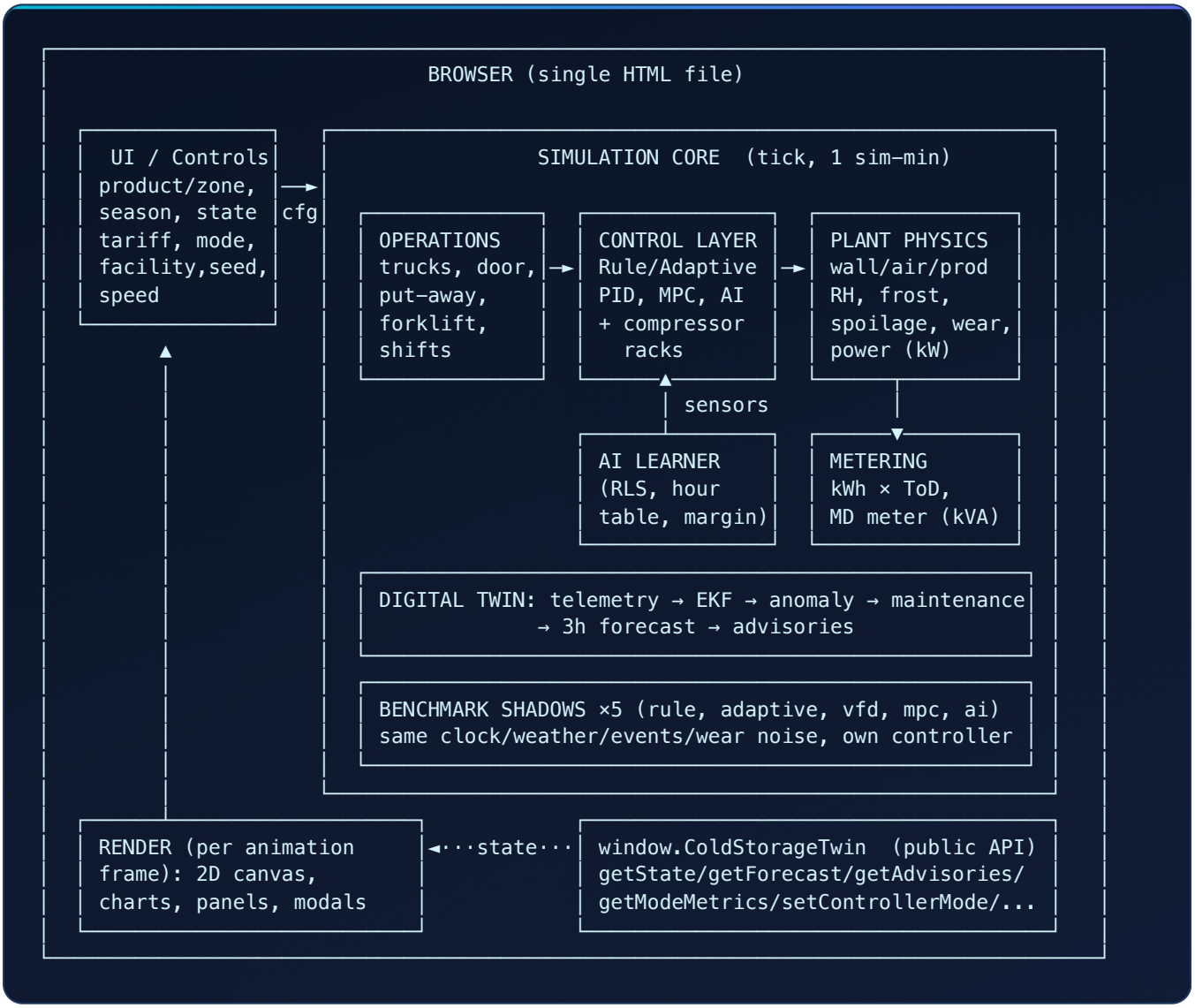
Companion
README.md · CONTROL_IO.md

Date
October 2026

0 Diagram legend

```
( Start / End )          terminal
[ Process ]              a step / function call
< Decision? >            a branch — the condition is written inside
{{ Data store }}         persistent state (state, shadowStates, cfg, learner …)
──►  ▼  ▲                control flow
··►                      data flow (read / write), not control
[D#]                     decision-maker tag, see §13 "Decision-maker register"
```

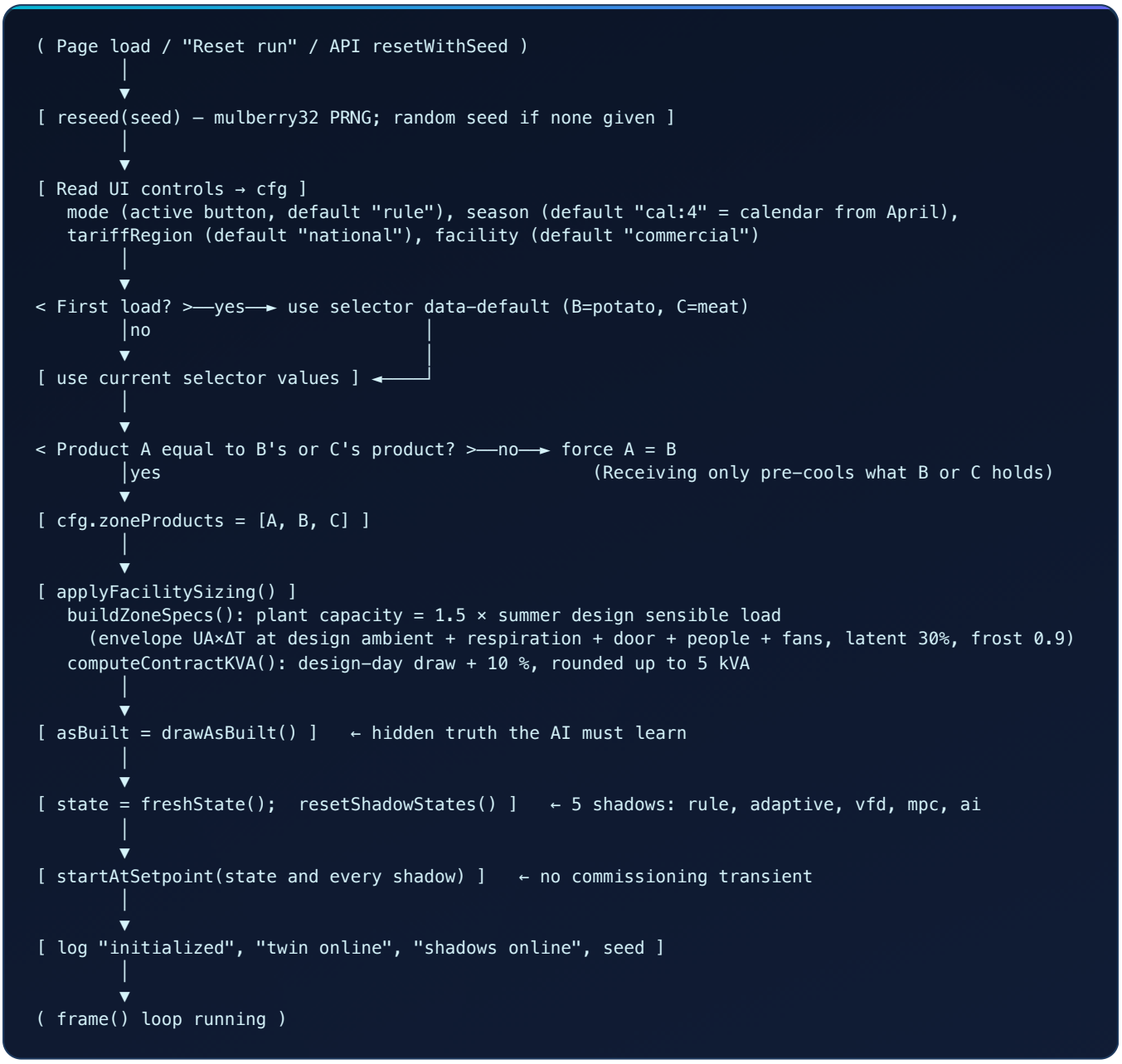
1 System context — layers



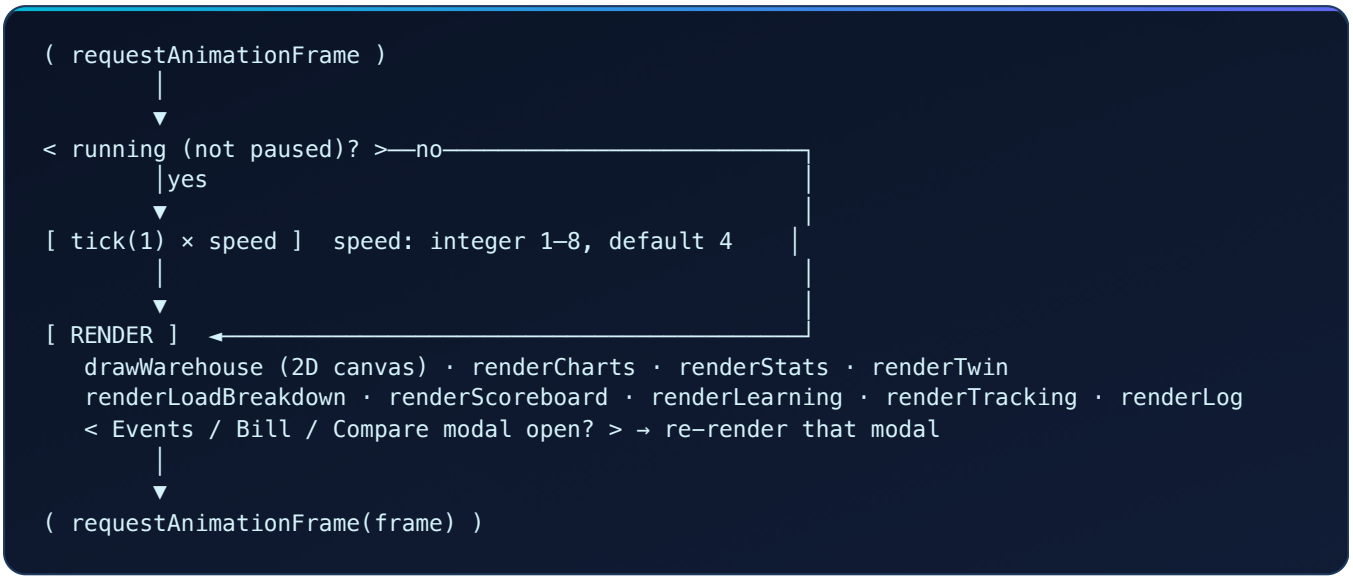
Core state objects

Store	Holds	Written by	Read by
`cfg`	mode, products per chamber, season, tariff region, facility, contract override	`setup()`, UI handlers, API	everything
`ZONE_SPECS[i]`	volume, envelope area, U, product mass, plant capacity W, door areas, idle kW	`applyFacilitySizing()` / `buildZoneSpecs()`	physics, controllers, twin
`asBuilt`	hidden deviations: UA ×1.10–1.35, capacity ×0.85–0.97, respiration ×0.8–1.3, sensor/probe bias	`drawAsBuilt()` (seeded)	**physics only** – never controllers
`state`	live, visualised run: temps, racks, telemetry, twin, events, ledger, forklift…	`tick()` and children	render, API
`shadowStates[mode]`	5 lightweight parallel runs – physics + control + metering only	`tick()` shadow loop	comparison modal, `getModeMetrics()`
`st.learner`	RLS θ, covariance P, hour-bias table, margins, delivery share, put-away delay	`learnerObserve()` (AI only)	`aiPlanZone()`

2 Startup flow – `setup(seed)`



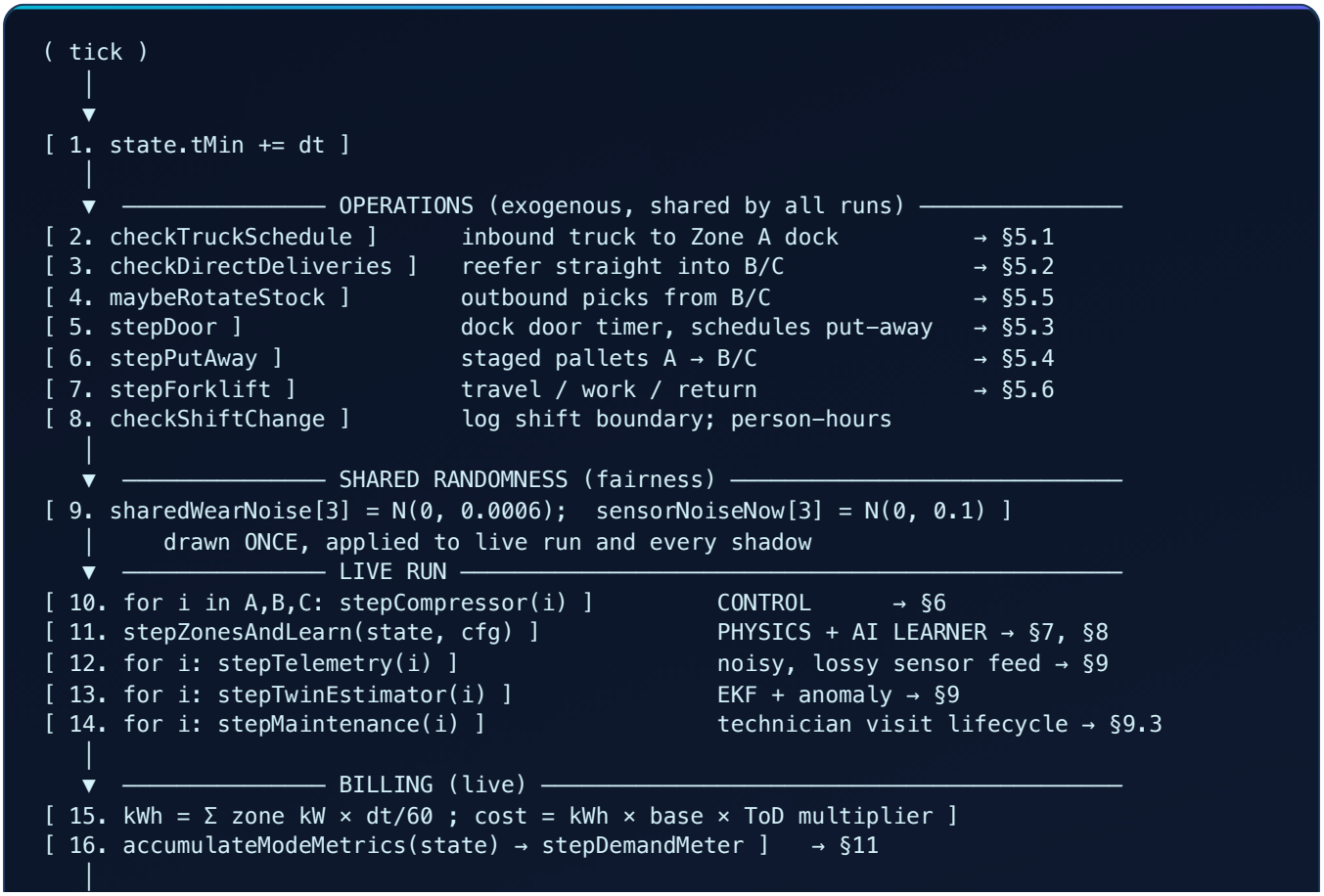
3 Main loop – `frame()` (per browser animation frame)

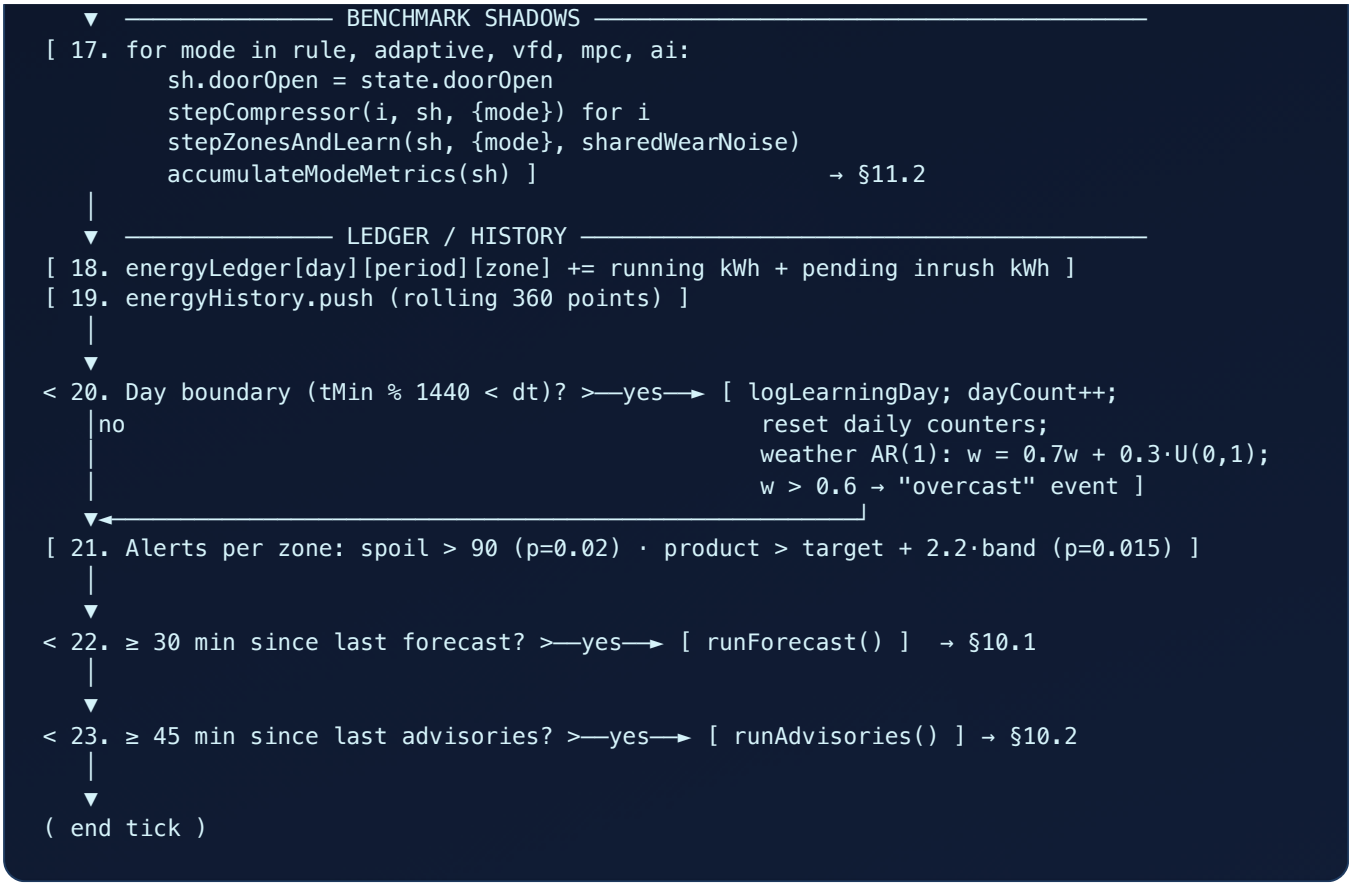


Approach: fixed 1-minute physics step, decoupled from render rate. Speed only changes how many ticks run per frame, so results are identical at any speed (and bit-identical per seed).

4 `tick(dt = 1 min)` – master sequence

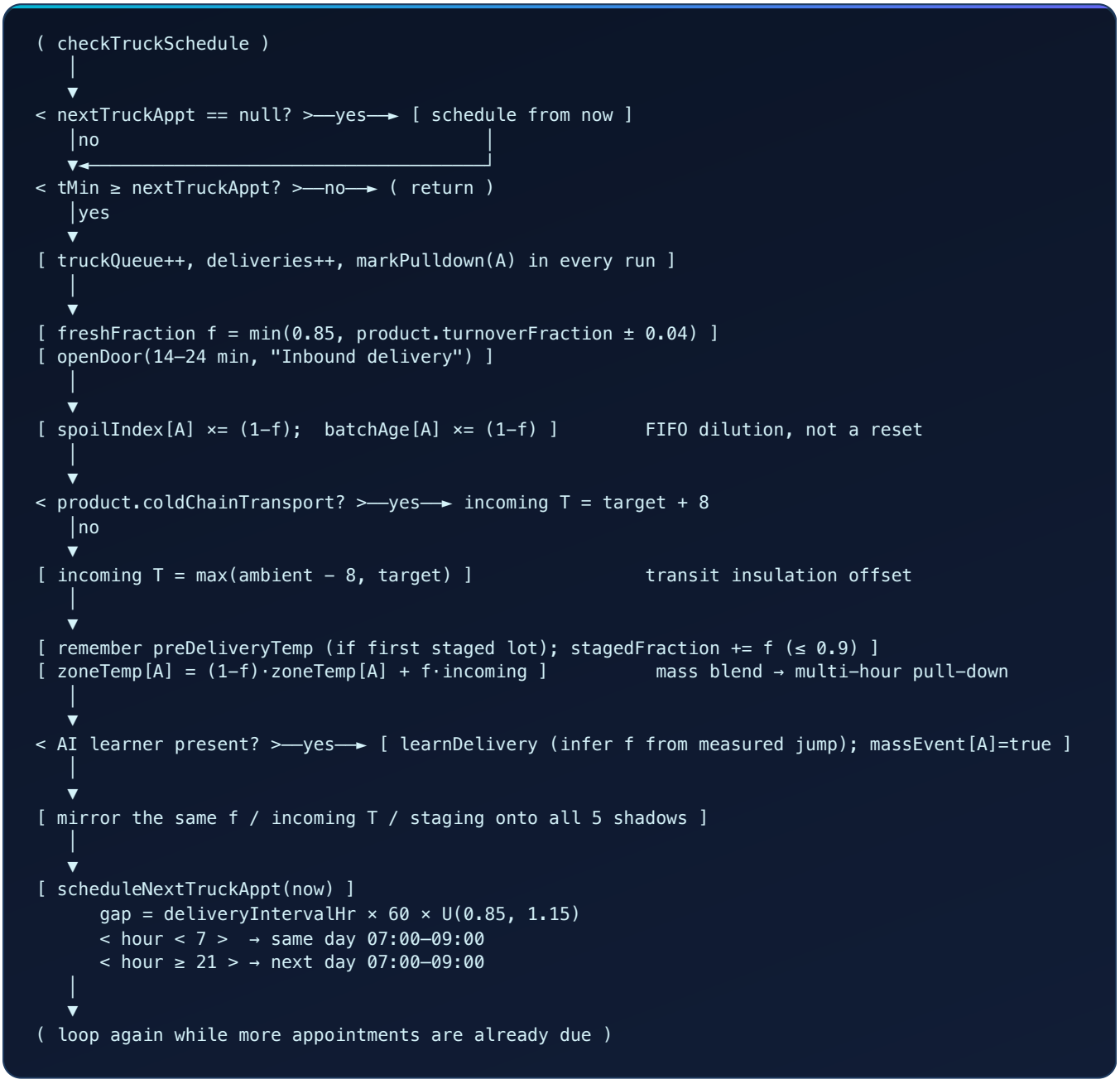
The order matters: operations events land first, then control decisions, then physics, then sensing/twin, then billing, then the shadows replay the same minute.




BENCHMARK SHADOWS
[17. for mode in rule, adaptive, vfd, mpc, ai:
sh.doorOpen = state.doorOpen
stepCompressor(i, sh, {mode}) for i
stepZonesAndLearn(sh, {mode}, sharedWearNoise)
accumulateModeMetrics(sh)] → §11.2
LEDGER / HISTORY
[18. energyLedger[day][period][zone] += running kWh + pending inrush kWh]
[19. energyHistory.push (rolling 360 points)]
< 20. Day boundary (tMin % 1440 < dt)? >—yes—► [logLearningDay; dayCount++;
no
reset daily counters;
weather AR(1): w = 0.7w + 0.3·U(0,1);
w > 0.6 → "overcast" event]
[21. Alerts per zone: spoil > 90 (p=0.02) · product > target + 2.2·band (p=0.015)]
< 22. ≥ 30 min since last forecast? >—yes—► [runForecast()] → §10.1
< 23. ≥ 45 min since last advisories? >—yes—► [runAdvisories()] → §10.2
(end tick)

5 Operations & scheduling

5.1 Inbound truck to Zone A – `scheduleNextTruckAppt` / `checkTruckSchedule` D1



5.2 Direct deliveries into B / C – `checkDirectDeliveries` D2

```
[ directStreamZones() = chambers j∈{B,C} whose product ≠ A's product
                        (and C excluded if C's product == B's) ]
   |
   ▼  for each such j
< directAppt[j] null? >──yes──► [ scheduleDirect(j) — same cadence/07–21 window rule as 5.1 ]
   |
   ▼
< tMin ≥ directAppt[j]? >──no──► next j
   |yes
   ▼
[ f = min(0.5, turnover × massA / massj);  incoming T as in 5.1 ]
[ for live + every shadow: blend zoneTemp[j], dilute spoil & age, learner.massEvent[j] ]
[ markPulldown(j); chamber door open 15–25 min (strip curtain) ]
[ reschedule ]
```

Why: frozen/chilled reefer loads would warm up if staged in a +4 °C receiving room, so they bypass A.

5.3 Dock door – `openDoor` / `stepDoor`

```
< doorOpen? >──no──► ( return )
   |yes
   ▼
[ doorTimer −= dt; doorOpenToday += dt ]
   |
   ▼
< doorTimer ≤ 0? >──no──► ( return )
   |yes
   ▼
[ close door; truckQueue-- ; close "door" event ]
   |
   ▼
< no put-away booked AND a delivery is staged (preDeliveryTemp ≠ null)? >──no──► ( return )
   |yes                                          (outbound door cycles do NOT trigger put-away)
   ▼
[ putAwayAt = now + U(60,150) min;  doorClosedAt = now ]
```

While open: infiltration on A's air = door area × 0.4 m/s × ρ·cp × ΔT × (1 − 0.55 air-curtain).

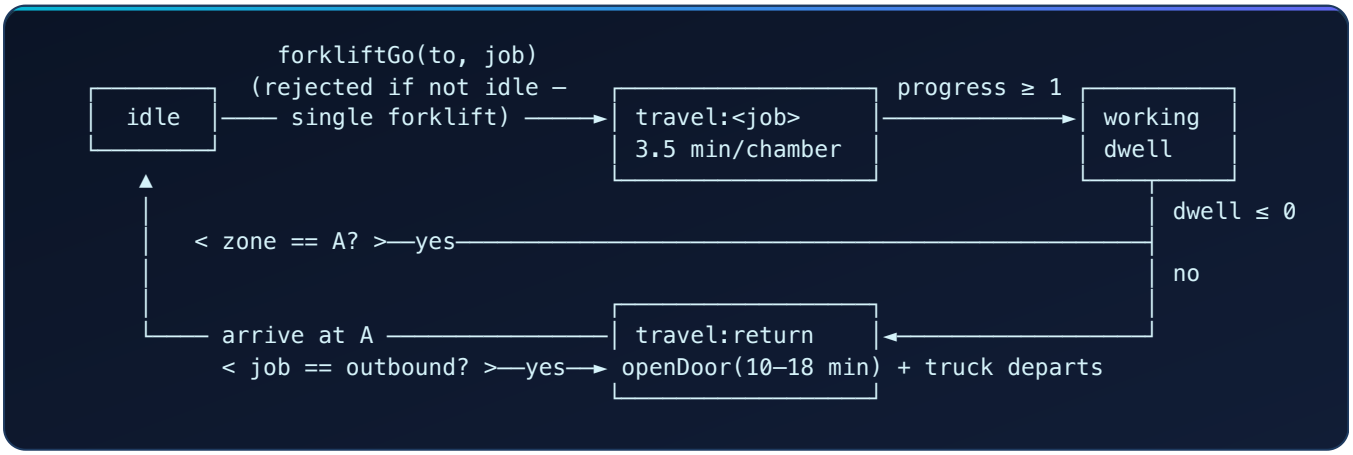
5.4 Put-away – `stepPutAway` D3

```
< putAwayAt set AND tMin ≥ putAwayAt? >──no──► ( return )
   │yes
   ▼
[ dest = putAwayDestination() ]
     < product(B) == product(A)? >──yes──► B
     < product(C) == product(A)? >──yes──► C
     else ──► B
   │
   ▼
[ learnPutAway(now − doorClosedAt) ]   (AI learners: EMA α=0.3 of real crew delay)
   │
   ▼
[ settle(run) for live + every shadow:
     stagedT = (zoneTemp[A] − (1−f)·preDeliveryTemp) / f      ← recover staged pallet temp
     zoneTemp[A] = 0.25·zoneTemp[A] + 0.75·preDeliveryTemp    ← 75 % cleared
     spoil/age[A] ×= 0.25
     m = min(0.5, 0.75·f·massA / massDest)
     zoneTemp[dest] = (1−m)·zoneTemp[dest] + m·stagedT         ← heat moves, doesn't vanish
     learner.massEvent[A, dest] = true;  clear staging ]
   │
   ▼
[ putAwayAt = null;  forkliftGo(dest, "put-away", 8–14 min dwell) ]
```

5.5 Outbound picks – `maybeRotateStock` D4

```
for i in B, C:
   cadence = max(240 min, min(base, 0.25 × refLife))     base: B = 4 days, C = 6 days
   < tMin % cadence < dt? >──no──► skip
        │yes
        ▼
   [ f = min(0.8, 0.9·turnover ± 0.03) ; spoil/age[i] ×= (1−f) in live + shadows ]   FIFO ships oldest
   [ forkliftGo(i, "outbound", 10–18 min) ]
```

5.6 Forklift state machine – `forkliftGo` / `stepForklift`



Side effects: inner doors of B/C open while the forklift works inside (strip curtain, 85 % blocked), and lighting runs (4 W/m²) while the chamber is occupied.

5.7 Shifts

Shift	Hours	Headcount	Sensible heat
Morning	07–15	4	A: n×130×1.3 W; B/C: n×35 W
Evening	15–21	3	same
Night	21–07	skeleton	40 W

6 Control layer – `stepCompressor(i, dt, st, md)`

One instance per chamber per run (live + 5 shadows). The **only manipulated variable** is cooling capacity, either a stage count (relay rack) or a capacity fraction (VFD-lead rack).

6.1 Top-level decision tree [D5 defrost, D6 safety, D7 mode]

```
( stepCompressor )
   |
   ▼
[ rack = ensureRack(st, md, i) ]  ← rebuilds if hardware type changed (relay ↔ vfd)
[ tickRackTimers: minRun, minRest, startPen, stageTimer -= dt; runHr += dt/60 ]
   |
   ▼
< defrosting? >──yes──► [ stopRack; capacity = 0; defrostTimer -= dt ]
   |no                          |
   |                            ▼
   |                 < defrostTimer ≤ 0? >──yes──► [ defrosting=false;
   |                           |no                   nextDefrostAt = now + interval
   |                           ▼                     (τ>30h → 480 min else 360 min) ]
   |                      ( return )                          |
   |                                                    ( return )
   ▼
[ policy = defrostPolicy(product) ]                                  [D5]
     < coil temp (target − TD) ≥ 0 °C? >──yes──► none   (e.g. tomato/banana/mango at 12–14 °C)
     < target > 0 °C? >──────────────yes──► off-cycle, 30 min, no heater
     else ──────────────────────────────► electric, 18 min, heater on
   |
   ▼
< policy ≠ none AND tMin ≥ nextDefrostAt? >──yes──► [ defrosting = true; timer = minutes;
   |no                                               piIntegral = 0; stopRack ] ──► ( return )
   ▼
[ airSetpoint = target − airSetpointTrim ]
     trim = min(½·band, respiration W ÷ air↔product coupling W/K)
[ over = measuredAirC − airSetpoint ]   ← lagged sensor + as-built bias + shared noise
   |
   ▼  ──── HIGH-HEAD-PRESSURE SAFETY (all modes) ────                [D6]
[ condensing = ambient + condenserApproach(mode, capacity) ]
< hpTripRemain > 0? >──yes──► [ stopRack; log 0 duty ] ──► ( return )
   |no
   ▼
< condensing > 55 °C AND compressor on? >──yes──► [ stopRack; hpTripRemain = 5 min; alert ] ──► ( return )
   |no
   ▼  ──── MODE DISPATCH ────                                        [D7]
< md.mode >
   ├── "vfd" ──────────────► §6.3 PI + feed-forward ──► rackContinuousStep(demand)
   ├── "mpc" | "ai" ───────► §6.4 planner (every 5 min) ──► rackContinuousStep(plannedCapacity)
   └── "rule" | "adaptive" ► §6.2 band/setpoint logic ──► rackRelayStep(over − spShift, band)
   |
   ▼
[ syncRackAggregates: capacityFraction = Σ on·cap·speed ; compressorOn = any unit on ]
[ lastHourOnMinutes.push(capacityFraction) — 60-slot ring ]
[ hourlyDutyProfile[hour] EMA, τ = 3 days (all modes, so switching into adaptive has history) ]
```

```
   │
   ▼
( return )
```

6.2 Rule-based & Adaptive band → relay rack

```
[ band = product.band ; spShift = 0 ]
   │
   ▼
< mode == adaptive? >──no──────────────────────────────────────────┐
   │yes                                                           │
   ▼                                                              │
[ recentDuty = mean(lastHourOnMinutes) ]                          │
[ historicalDuty = hourlyDutyProfile[hourNow] ]                   │
[ blended = 0.6·recent + 0.4·historical ]                         │
   │                                                              │
   ▼                                                              │
< blended > 0.55? >──yes──► band ×0.7  (tighten under load)       │
   │no ──────────────────► band ×1.25 (loosen when idle)          │
   ▼                                                              │
< tariff period >                                                 │
   ├─ peak ──────────────► spShift = +0.35·band  (coast on thermal mass)
   ├─ offpeak / solar ──► spShift = −0.30·band  (pre-cool cheaply)
   └─ normal ────────────► 0                                      │
   ▼                                                              │
[ clamp band to [0.5, 1.5] × base band ]          hard safety envelope
[ bound spShift so the thermostat cycle stays ≥ 0.2·band inside    │
   the product band (lowest ≥ −0.8·band, highest ≤ +0.8·band) ]   │
   │                                                              │
   ▼◄─────────────────────────────────────────────────────────────┘
[ rackRelayStep(over − spShift, band) ]
```

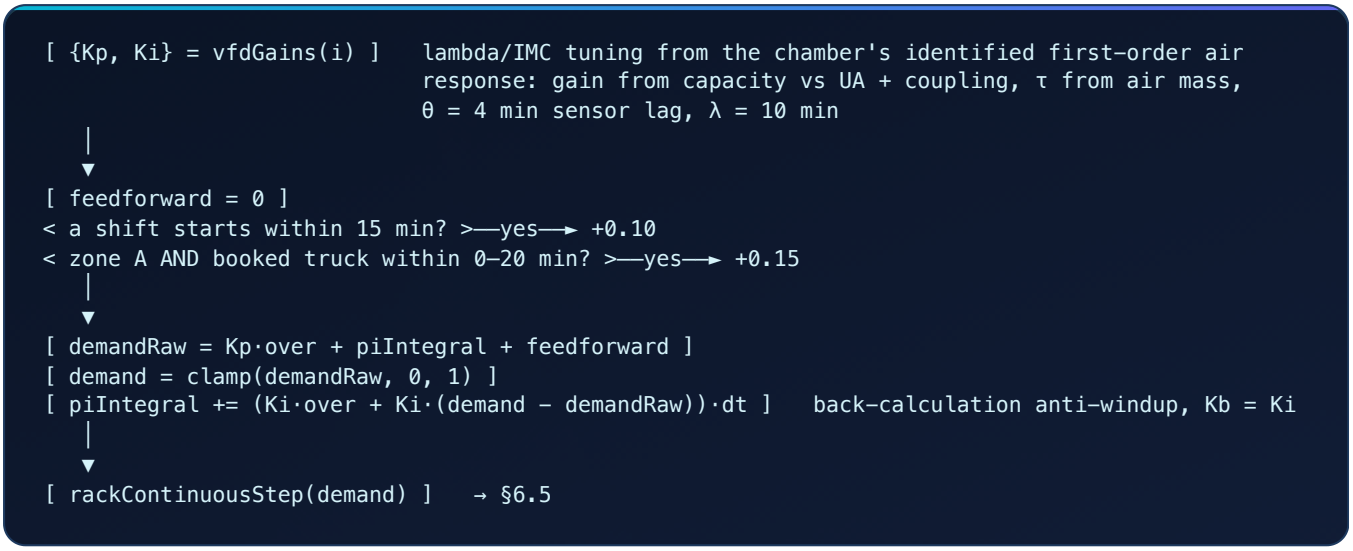
`rackRelayStep` **– step thermostat ladder centred on the setpoint**

```
n = units in rack
cutIn(k)  = band·((k+1)/n − ½)
cutOut(k) = cutIn(k) − band·(½ + ½/n)

[ k = stageTarget ]
while k < n AND over > cutIn(k)     : k++          stage up
while k > 0 AND over < cutOut(k−1)  : k−−          stage down
[ stageTarget = k ]
   │
   ▼
< running < k AND stageTimer ≤ 0? >──yes──► [ u = pickStart (least run-hours, canStart) ]
   │no                                          │ found?
   │                                           ▼
   │                                  [ startUnit(u); stageTimer = 2 min ]
   ▼
< running > k? >──yes──► [ u = pickStop (most run-hours, minRun ≤ 0) → stopUnit(u) ]
```

`canStart(u)` = off **and** minRest ≤ 0 **and** ≥ 10 min since its last start (≤ 6 starts/h).

6.3 PID / VFD


[{Kp, Ki} = vfdGains(i)]
lambda/IMC tuning from the chamber's identified first-order air response: gain from capacity vs UA + coupling, τ from air mass, θ = 4 min sensor lag, λ = 10 min
[feedforward = 0]
< a shift starts within 15 min? >—yes→ +0.10
< zone A AND booked truck within 0–20 min? >—yes→ +0.15
[demandRaw = Kp·over + piIntegral + feedforward]
[demand = clamp(demandRaw, 0, 1)]
[piIntegral += (Ki·over + Ki·(demand − demandRaw))·dt]
back-calculation anti-windup, Kb = Ki
[rackContinuousStep(demand)]
→ §6.5

6.4 MPC / AI planner – `aiPlanZone` + `aiRollout` D8

```
< tMin ≥ aiNextPlanAt[i]? >──no──► [ hold aiPlannedCapacity[i] ] ──► rackContinuousStep
   │yes
   ▼
[ ctx = aiContext() ]  horizon 720 min in 15-min steps (48 steps):
     ambient forecast (seasonal shape only), ToD multipliers, shift heat, tail price
   │
   ▼
< mode == ai? >──yes──► L = learner (learned UA, heat bias, hour table, delivery share,
   │no                  put-away delay, weather offset, per-chamber margin)
   ▼
   L = null (nameplate everything, margin 15 %)
   │
   ▼
[ Build per-step exogenous schedule ]
   • Zone A truck: door coefficient during [appt, appt+19], stock blend at appt (share f)
   • Put-away: at WMS-booked putAwayAt, else door-close + paDelay (learned or 105 min)
       → A returns toward pre-delivery temp; dest gets staged mass inflow
   • Direct delivery into B/C: blend + inner-door heat
   • Defrost minutes per step (aiDefrostMinutes)
   • Lights (A by roster), clear-sky roof sol-air, latent fraction from room RH
   • Tp0 = 15-min filtered probe reading (not truth)
   • Demand-charge thresholds: mdBilled = max(month MD, floor), mdExcessFrom = max(month MD, contract)
   │
   ▼
[ othersKW[k] = other chambers' latest planned kW (Gauss-Seidel coordination), else their current kW ]
   │
   ▼
[ Decision vector = 7 stages over 12 h  (30m,30m,1h,2h,2h,3h,3h) ]
[ LEVELS = {0, rackMin (VFD lead at 25 %), 0.2, 0.3, 0.45, 0.6, 0.8, 1.0 > rackMin} ]
   │
   ▼
[ Candidate seeds: previous plan shifted & snapped + every constant level ]
[ evaluate each with aiRollout → keep cheapest ]
   │
   ▼
[ Coordinate descent: up to 2 sweeps; for each stage try every level;
  keep any strict improvement; stop early if a sweep improves nothing ]
   │
   ▼
[ store plan {t0, levels, kW, cost}; queue 1h & 6h forecast checks (scoreForecasts) ]
[ aiNextPlanAt = now + 5 min ]
   │
   ▼
( return best[0] – only stage 1 is applied: receding horizon )
```

`aiRollout` – cost of one candidate plan (per 15-min step k)

```
split = rackSplit(n, level)                    ← lead speed + lags the rack would settle into
Tair  = quasi-steady air balance with coil output ∝ (Tair − Tcoil), saturating at [0, 1.5]
Tp    += (coupling·(Tair − Tp) + respiration)·dt / Cprod
apply blend / put-away / inflow events at their step
frost grows with load (coil < 0 °C); defrost melts

kW   = idle + evap fans + lights + lead kW + lags kW + condenser fans (+ heater)
COST =  Σ kW·dt·base·ToD[k]                                          energy
     +  rate·max(0, peak kVA (incl. othersKW) − mdBilled)             new maximum demand
     +  rate·(2−1)·max(0, peak kVA − mdExcessFrom)                    over-contract penalty
     +  restart cost (cold-start loss) per 0→on lead / extra lag       cycling
     +  1h·ratedKW·base·(Δlevel)²                                      move suppression
     +  Σ dev·15·(pull-down? spoilW : qualityW)                        product outside band − margin
     +  Σ (airLow − Tair)·15·qualityW  if air < target − band − 3      supply-air low limit
     +  (Tp_end − Tp0)·Cp / copRef × tail price                        terminal value of stored cold
```

Decision rules inside the rollout:

Condition	Branch
`Tp > hi` while a delivery is still pulling down	priced at **shelf-life cost** `spoilW` (₹ value × Q10 × shelf life)
`Tp` outside `[lo, hi]` otherwise	priced at **quality penalty** = 60 min of full rated power per °C·min
`hi/lo`	band shrunk by margin (15 % nameplate, or learned per chamber for AI)

6.5 VFD-lead rack – `rackContinuousStep(demand)`

```
lead = unit 0 (VFD, 1.4× a lag's size), lags = fixed-speed units 1..n-1
[ leadTarget = (demand − lagsOn·lagCap) / leadCap ]
   │
   ▼
< n > 1? >──no──────────────────────────────────────────────┐
   │yes                                                    │
   ▼                                                       │
[ upTimer   += dt  if leadTarget > 1 AND lead ≥ 97 %, else 0 ]
[ downTimer += dt  if lags on AND leadTarget < 25 %, else 0 ]
   │                                                       │
< upTimer ≥ 3 min AND stageTimer ≤ 0? >──yes──► start least-hours lag; leadTarget −= lagCap/leadCap
< downTimer ≥ 5 min? >──────────────yes──► stop most-hours lag;  leadTarget += lagCap/leadCap
   │                                                       │
   ▼◄──────────────────────────────────────────────────────┘
< lead off? >──yes──► < leadTarget ≥ 25 % OR lags on, AND canStart(lead)? >──yes──► startUnit(lead) at 25 %
   │no
   ▼
< no lags AND leadTarget < 12.5 % AND minRun done? >──yes──► stopUnit(lead)   (hysteresis)
   │
   ▼
< lead on? >──yes──► [ speed → clamp(leadTarget, 0.25, 1), slew ≤ 6 % of zone capacity/min ]
```

6.6 Unit start/stop – `startUnit` / `stopUnit`

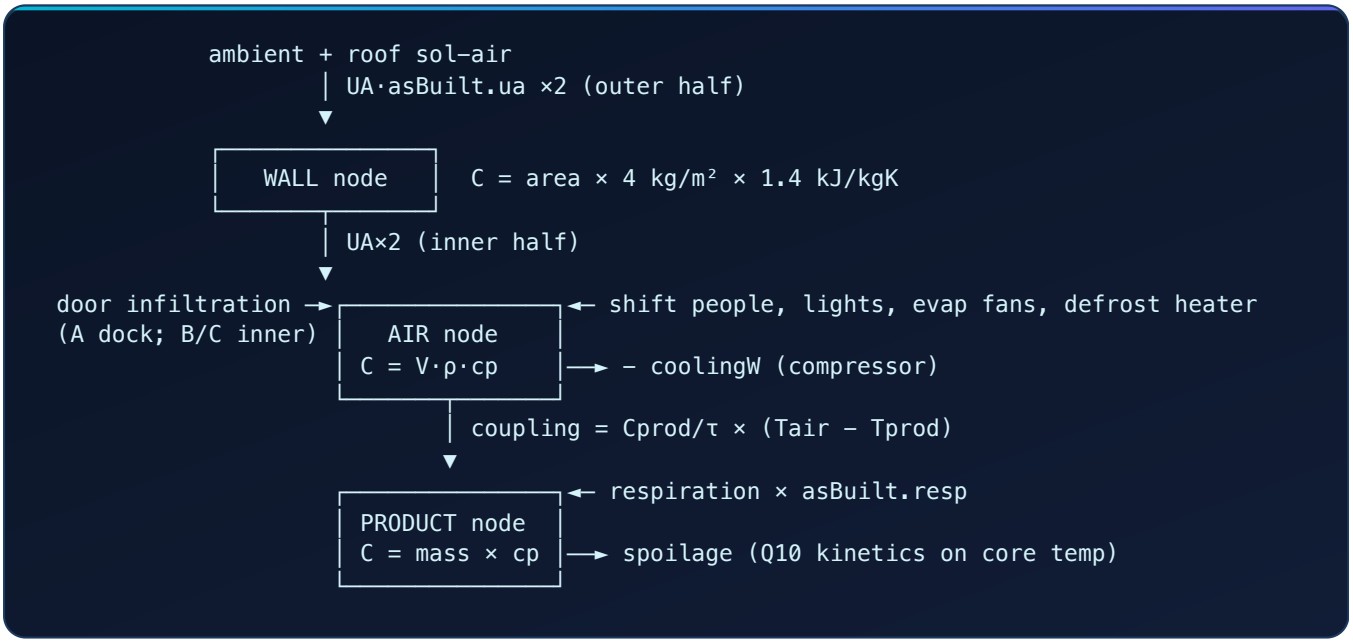
```
startUnit:
  on, speed = (vfd ? 0.25 : 1), minRun = 5 min, startPen = 3 min (COP × 0.82)
  starts++, cyclesStarted++ (benchmark)
  registerCompressorEngage:
      cumulativeCycles += 1/n
      < not VFD? >──yes──► contactorOps++
      < live AND contactor wear ≥ 90 % AND not flagged/visiting? >──yes──► dispatchMaintenance("contactor")
  starter = VFD soft start (1.5× / 3 s) | star-delta if > 11 kW (~1.9×) | DOL (~5.8×)
  chargeStartEnergy → one-time kWh at current ToD rate

stopUnit:  off, speed 0, minRest = 3 min
```

7 Plant physics – `stepZonesAndLearn` → `stepZone(i)`

```
( stepZonesAndLearn(st, md) )
   |
   ▼
[ clear pulldownZ[i] once measured product ≤ target + band ]
   |
   ▼
for each zone i:
   [ pre = {air, prod} (AI only) ] → [ stepZone(i) ] → < mode == ai? >──yes──► learnerObserve(i, pre)  §8
   |
   ▼
< predictive mode (mpc/ai)? >──yes──► [ probeFilt += (measuredProduct − probeFilt)·dt/15 ]
   |                                  [ scoreForecasts ]  (1h / 6h prediction accuracy)
   ▼
( return )
```

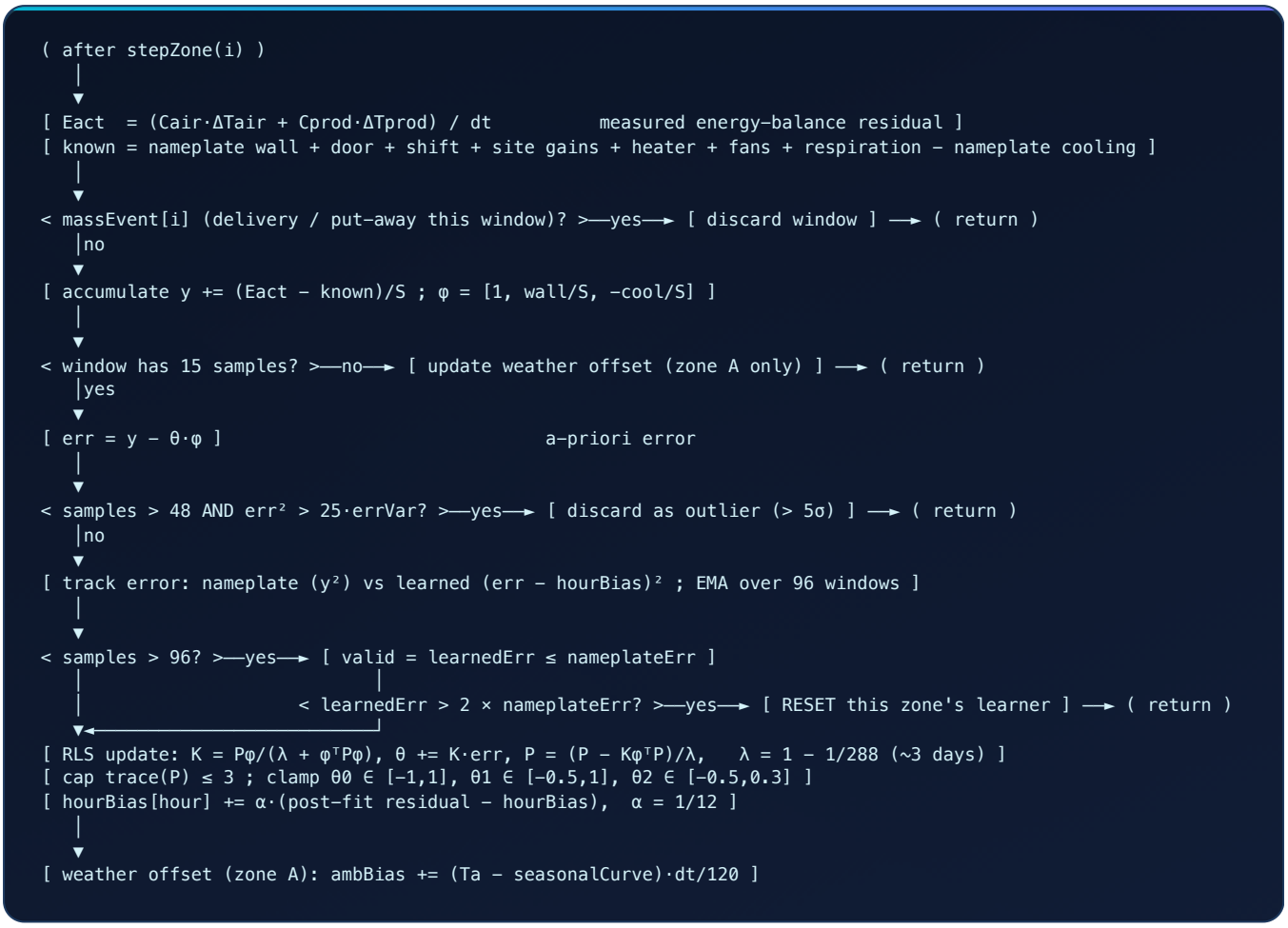
`stepZone(i)` – three coupled thermal nodes



Step-by-step:

```
[ wall:    dWall = (UAhalf·(Tout − Twall) − UAhalf·(Twall − Tair))·dt / Cwall ]
[ door:    < doorOpen AND zone has dock door? > → area·0.4·pcp·ΔT·(1 − 0.55)
           < inner door open (forklift inside)? > → innerArea·0.4·pcp·max(0, ΔT_corridor)·0.15 ]
[ lights:  < chamber occupied? > → roofArea × 4 W/m² ]
[ cooling: < defrosting? > → 0
           else capacityW × frostPenalty(≤ −60 %) × trueCapMult(wear) × capacityFraction
                × (1 − latentFraction(roomRH)) × asBuilt.cap × coilFactor((Tair − Te)/(target − Te), 0..1.5) ]
[ air  += (wallIn + door + people + lights + heater + evapFan − cooling − coupling)·dt / Cair ]
[ prod += (coupling + respiration·asBuilt.resp)·dt / Cprod ]
   │
   ▼
[ POWER (kW) ]
   < defrosting? >──yes──► heater kW
   │no
   ▼
   for each running unit:  coolW·cap·speed / (COP × frostCOP(≤ −35 %) × eff × startEff)
       COP     = cycleCOP: 0.55 × Tevap / (Tcond − Tevap), Tcond = max(25, ambient + approach)
       approach= 12 °C fixed (relay) | 12 × clamp(capFrac, 0.35, 1) (VFD rack: floating head)
       eff     = VFD lead: vfdEfficiencyMult(speed) × 0.97 drive loss ; lag: 1
       startEff= 0.82 within 3 min of that unit's start
   + condenser fans 4 % of capacity × (staged | floating)
   + evaporator fans + idle (controls, crankcase) + lights
   │
   ▼
[ WEAR ]   runtime (equivalent full-load h) += dt/60 × capFrac
           trueCapMult = max(0.8, 1 − runtime·0.10/16000 − cycles·0.10/50000 + sharedNoise·√dt)
[ SENSOR ] drift random walk; < p=0.0007·dt AND |drift|>0.3 > → recalibrate ×0.15
           zoneSensor lags air with τ = 4 min
[ RH ]     + transpiration (respiring 0.01 | other 0.006)·(1−rh)
           + door exchange toward outdoor RH
           − coil condensation 0.02·capFrac·max(0, rh − 0.55)     clamp [0.25, 0.99]
[ FROST ]  < on AND not defrosting AND Tevap < 0? > → += 0.0006·rh·dt·capFrac
           < defrosting? > → melt (electric 0.02/min | off-cycle 0.004·max(0.5, Tair))
[ SPOIL ]  rate = (100 / refLife·60) × Q10^(max(0, Tprod − target)/10)
```

8 AI learner – `learnerObserve` (mode "ai" only) D9



Other learned quantities:

Learned	How	Used by planner as
UA multiplier, steady heat bias	RLS θ1, θ0	`z.UA`, `z.extraW`
Hour-of-day heat	EMA table on residual	`z.extraW[k]` per hour
Delivery share	`learnDelivery`: f = (after − before)/(incoming − before), EMA 0.3	stock blend size
Put-away delay	`learnPutAway`: door-close → put-away, EMA 0.3	put-away step
Weather offset	EMA of today vs seasonal curve	ambient bias decaying e^(−t/12h)
Per-chamber margin	widened on out-of-band outcomes (not during pull-down), relaxes slowly	`marginFrac`
Capacity θ2	RLS	**diagnostic only** – closed-loop bias 3-4 % low, so not fed to planner
Validity gate	learned error ≤ nameplate error	`aiExtraHeat` returns 0 when invalid

9 Digital twin — sensing, estimation, anomaly, maintenance

9.1 Telemetry — `stepTelemetry(i)`

```
< rng < 0.015 (dropout)? >──yes──► [ telemetryStale += dt; hold last value ] ──► ( return )
   |no
   ▼
[ raw = zoneSensor (4-min lagged air) + sensorDriftBias + N(0, 0.15) ]
[ telemetry = round(raw, 0.1 °C) ;  stale = 0 ]
```

9.2 EKF estimator — `stepTwinEstimator(i)` D10

```
State x = [ airTemp, capMult, uaMult ]      measurement: telemetry only (H = [1,0,0])
   |
   ▼
[ PREDICT ]
   netW = UA·uaMult·(Tout − x0) + door·(Ta − x0) + people + gains + heater + fans
          − capCoef·capMult + coupling·(twinProduct − x0)
   xPred = [x0 + a·netW, capMult, uaMult]                parameters: random walk
   F     = Jacobian at x ;  Ppred = F·P·Fᵀ + Q
   twinProduct += (coupling + respiration)·dt / Cprod    derived, outside the filter
   twinPredSensor lags xPred[0] with τ = 4 min           (matches the real sensor housing)
   |
   ▼
[ residual EMA = 0.9·residual + 0.1·|telemetry − twinPredSensor| ]
   |
   ▼
< telemetry fresh (stale == 0)? >──no──► [ x = xPred ; P = Ppred ]   (don't trust held values)
   |yes
   ▼
[ UPDATE ]  S = Ppred00 + R(0.023);  K = Ppred[:,0]/S;  x = xPred + K·err;  P = Ppred − K·Ppred[0,:]
            clamp capMult ∈ [0.4, 1.05], uaMult ∈ [0.5, 2.0]
   |
   ▼
< no baseline yet AND runtime > 40 h? >──yes──► [ twinBaseline = {cap, ua} ; log commissioning ]
   |
   ▼
[ zCap = (baseline.cap − capMult)/√Pcap ;  zUa = |uaMult − baseline.ua|/√Pua ]
   |
   ▼
< baseline set AND zCap > 3 AND not flagged/visiting? >──yes──► [ flag; dispatchMaintenance("service") ]
   |
   ▼
< baseline set AND residual > 4×residualBaseline AND zCap < 1.5 AND zUa < 1.5 >──yes──►
                                    [ flag; dispatchMaintenance("calibration") ]
                                    (residual high but equipment normal ⇒ sensor drift)
```

9.3 Maintenance lifecycle – `dispatchMaintenance` / `stepMaintenance` D11

```
dispatch(kind ∈ {service, calibration, contactor})
   < visit already active? >──yes──► ignore
   < hour < 14? >──yes──► ETA now + 1–4 h      ──no──► next day 08:00–11:00
        │
        ▼
┌─────────────┐  tMin ≥ ETA  ┌─────────────┐  tMin ≥ endsAt  ┌─────────────┐  +20 min  ┌──────┐
│ scheduled   │─────────────►│ on site     │────────────────►│ completed   │──────────►│ none │
└─────────────┘              │ 45–90 min   │                 └─────────────┘           └──────┘
                             └─────────────┘                        │
                                                                    ▼
                                   < kind >
                                     ├─ service     → compRuntimeHr ×0.15 (wear reset), unflag
                                     ├─ contactor   → most-worn unit contactorOps = 0, unflag
                                     └─ calibration → sensorDriftBias = 0, unflag
```

10 Forecast & advisories

10.1 `runForecast()` – every 30 sim-min

```
[ start from twin estimates (air, product), frost; duty = last-hour mean capacity per zone ]
for t = 0..180 min, record every 5, integrate in 1-min substeps:
   wall    = UA·estUaMult·(Tout − air)
   cooling = capacity·duty·frostPenalty·estCapMult·(1 − latent)·coilFactor
   air     += (wall + 150 + evapFan − cooling − coupling)/Cair
   product+= (coupling + respiration)/Cprod
→ state.forecast = [{t, air[3], temp[3]}]       (ambient held flat — known simplification)
```

10.2 `runAdvisories()` – every 45 sim-min D12

```
advisories = aiPlanAdvisories()          ← reads live MPC/AI plan, else the AI shadow's plan
   • facility 12-h peak kVA vs max(month MD, floor)  → "demand guard" ok / warn
   • per zone: Peak window starts within 4 h AND now-level > peak-avg + 8 % → "banking cold"
+ rule checks per zone:
   < duty > 75 % AND frost > 45 % >                    → warn: pull an early defrost
   < estCapMult < 0.9 × baseline cap >                 → bad: schedule technician
   < forecast product at +1 h > target + 1.8·band >    → warn: pre-cool now
   < next 2 h becomes Peak AND product < target − 0.2·band > → ok: coast through peak
→ state.advisories = first 6
```

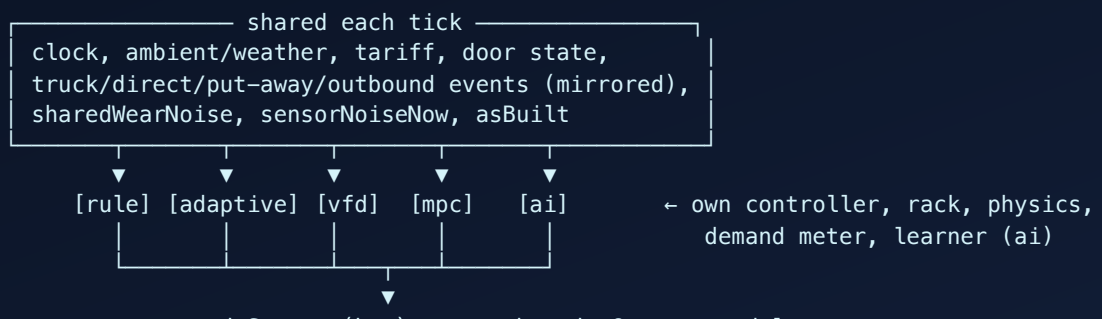
11 Metering, billing & benchmarking

11.1 Energy & maximum demand

```
per tick (live and each shadow):
   kWh  = Σ zone kW × dt/60
   cost = kWh × baseTariff × ToD multiplier(period)   (+ one-time start energy charged at start)
   accumulateModeMetrics:
       minutesTracked, costByPeriod[period], degMinOutside, zoneMinutesInBand,
       minutesInBand (all 3 zones in band), spoilStart/spoilLast
   stepDemandMeter:
       blockKWh += kWh
       < block ≥ mdBlockMin (15/30 by state)? >──yes──► kVA = kWh/h ÷ 0.92 PF ;
                                                        monthMax = max(monthMax, kVA)
       < new 30-day billing month? >──yes──► close month:
            rs = rate·max(MD, floor·contract) + rate·(2−1)·max(0, MD − contract)

Bill = energy ₹ + demand ₹ (closed months + running month pro-rated)
```

11.2 Benchmark shadows – fairness rules



Shadows skip the twin, telemetry dropouts and the event log on purpose: they only need ground-truth physics + control + billing. **Rule for new code:** any new random term in `stepZone` / `stepCompressor` must be drawn once in `tick()` and shared, otherwise it adds noise to the comparison between controllers.

12 External integration – `window.ColdStorageTwin`

Method	Direction	Returns / effect
`getState()`	read	per-zone telemetry, twin air/product estimates, est. capacity/UA %, residual, compressor, defrost, frost, RH, spoilage, anomaly (`capacity_drift` / `sensor_drift`); facility power, energy, cost, MD, contract, tariff, mode, forklift, cumulative counters
`getForecast()`	read	3-h twin forecast
`getAdvisories()`	read	current advisory list

Method	Direction	Returns / effect
`getModeMetrics()`	read	deep copy of all 5 shadow ledgers + bill, % in safe range, peak kVA
`getProducts()`	read	product catalogue
`setProductProfile(key, zone?)`	write	one chamber, or all
`setControllerMode(mode)`	write	switches the visualised run's controller
`triggerTruckArrival()`	write	opens dock 16 min
`getSeed()` / `resetWithSeed(seed)`	read / write	reproducible replay

Real-hardware seam: replace `stepTelemetry` (and the controllers' sensor reads) with a live feed, and replace `startUnit` / `stopUnit` /lead speed with contactor / VFD commands. Everything from the estimator up stays the same.

13 Decision-maker register

#	Decision maker	Decides	Inputs	Cadence	Approach
D1	`scheduleNextTruckAppt`	next inbound truck time	product delivery interval, clock	after each arrival	interval × U(0.85–1.15), forced into 07:00–21:00
D2	`checkDirectDeliveries`	which chambers get reefer deliveries, when	products per chamber	per tick	products that differ from A's get their own delivery schedule
D3	`putAwayDestination` / `stepDoor`	where & when staged pallets go	products per chamber, door close	door close (+60–150 min)	same-product chamber, B first
D4	`maybeRotateStock`	outbound pick time & share	shelf life, base cadence	per tick	min(base, ¼ shelf life), FIFO
D5	`defrostPolicy`	defrost method & duration	coil temp, target	each defrost interval (6/8 h)	none / off-cycle 30 min / electric 18 min
D6	High-pressure cut-out	force rack off	ambient + condenser approach	every min	> 55 °C condensing → 5-min lockout
D7	`stepCompressor` mode switch	which control law	`md.mode`	every min	dispatch table
D7a	Rule (`rackRelayStep`)	stages on	lagged air sensor, band, trim	every min	centred step-thermostat ladder

#	Decision maker	Decides	Inputs	Cadence	Approach
D7b	Adaptive	band width + setpoint shift	duty (60 min + hour profile), ToD period	every min	rules + safety clamp
D7c	PID	capacity 0–1	air error, roster, booked truck	every min	lambda-tuned PI + feed-forward + back-calc anti-windup
D8	MPC / AI (`aiPlanZone`)	capacity for next 5 min (12-h plan)	filtered probe, schedules, tariff, MD, other zones' plans, learned model (AI)	every 5 min	economic MPC, 7 stages, coordinate descent, receding horizon
D8a	`rackContinuousStep`	lead speed, lag staging	demand	every min	3-min stage-up / 5-min stage-down delays, 25 % floor, slew 6 %/min
D8b	`pickStart` / `pickStop`	which unit	run-hours, timers	on stage change	lead-lag rotation by run-hours
D9	`learnerObserve`	model parameters; reset or keep	measured energy balance	15-min windows	RLS with forgetting + gates (mass event, outlier, validation)
D10	`stepTwinEstimator`	capacity/UA estimate; service vs calibration alarm	telemetry	every min	EKF + z-scores vs commissioning baseline
D11	`dispatchMaintenance`	visit ETA & action	alarm kind, hour	on alarm	same day if before 14:00, else next morning
D12	`runAdvisories`	operator recommendations	twin, forecast, plans, tariff	every 45 min	AI-plan readout + rule checks

14 Key constants (tuning surface)

Constant	Value	Meaning
tick	1 sim-min	physics/control step
`MIN_RUN_MIN / MIN_REST_MIN / MIN_START_INTERVAL_MIN`	5 / 3 / 10 min	anti-short-cycle per unit
`STARTUP_COP_PENALTY`, `STARTUP_PENALTY_MIN`	0.82, 3 min	cold-start loss
`INTERSTAGE_DELAY_MIN`, `STAGE_UP/DOWN_DELAY_MIN`	2, 3, 5 min	rack staging
`VFD_MIN_SPEED`, `VFD_LEAD_SIZE_RATIO`, `VFD_DRIVE_LOSS`	0.25, 1.4, 0.97	VFD rack
`CONDENSER_APPROACH_FULL_C`, `..._MIN_APPROACH_FRAC`	12 °C, 0.35	head-pressure floating
`HP_TRIP_CONDENSING_C`	55 °C	safety trip

Constant	Value	Meaning
`AI_REPLAN_INTERVAL_MIN`, `AI_STEP_MIN`, `AI_HORIZON_MIN`	5, 15, 720	MPC timing
`AI_STAGE_EDGES`	30m,30m,1h,2h,2h,3h,3h	plan stages
`AI_BAND_MARGIN_FRAC`	0.15	nameplate safety margin
`AI_QUALITY_PENALTY_FULLPOWER_MIN`	60	₹ weight per °C·min
`AI_AIR_LOW_MARGIN_C`	3 °C	supply-air low limit
`AI_MOVE_SUPPRESSION_H`	1 h	step-change cost
`MD_EXCESS_PENALTY_MULT`, `SITE_POWER_FACTOR`	2, 0.92	demand billing
`LEARN_WINDOW_MIN`, `LEARN_FORGET`, `LEARN_HOUR_ALPHA`	15, 1−1/288, 1/12	learner
`UNIT_TARGET_CAPACITY_W`, `MAX_UNITS_PER_ZONE`	100 kW, 4	rack sizing
`CONTACTOR_RATED_OPS`	100,000	contactor life
Defrost interval	360 min ($\tau \le 30$ h) / 480 min	schedule